

# DESIGN JUSTIFICATION REPORT

ECE 410/510 — Hardware for Artificial Intelligence and Machine Learning  
Portland State University | Spring 2026

**Project: BF16 Conv2d Accelerator — 32x32 Weight-Stationary Systolic Array**  
**PDK: SAED14nm RVT | Tool: Cadence Genus 17.14-s037\_1 | Clock: 500 MHz**

## INDEX

| Section No. | Title                     | Page No. |
|-------------|---------------------------|----------|
| 1.          | Problem and Motivation    | 2-3      |
| 2.          | Roofline Analysis         | 4-5      |
| 3.          | Precision and Data Format | 6        |
| 4.          | Dataflow and Architecture | 7-8      |
| 5.          | Hardware Interface        | 9        |
| 6.          | Verification              | 10-11    |
| 7.          | Synthesis Results         | 12-13    |
| 8.          | Benchmark Results         | 14-15    |
| 9.          | What did not work         | 16-17    |

## **SECTION 1: PROBLEM AND MOTIVATION**

The target application is a hybrid anemia-detection model (ResNet18 backbone + AttentionFusion MLP) trained on the AneRBC peripheral blood smear dataset (12,000 images: 6,000 healthy, 6,000 anemic). The model was profiled running on an Intel Core i5-10210U laptop CPU (Windows 11, PyTorch 2.11.0, CPU-only) over 10 full training epochs, each epoch processing 9,600 training images in 300 batches of 32. Wall-clock timing (time.perf\_counter, full pipeline including data loading, feature extraction, validation, GradCAM, and logging):

### **1.1 Wall-Clock Timing (time.perf\_counter, full pipeline)**

| Run    | Time     |
|--------|----------|
| Run 1  | 2762.6 s |
| Run 2  | 2670.4 s |
| Run 3  | 2595.1 s |
| Run 4  | 2537.6 s |
| Run 5  | 2552.6 s |
| Run 6  | 2497.8 s |
| Run 7  | 2508.4 s |
| Run 8  | 2481.1 s |
| Run 9  | 2860.9 s |
| Run 10 | 2719.2 s |

**Median wall-clock: 2573.9 s (42.9 minutes) per epoch**

**Mean: 2618.6 s**

**Std deviation: 128.8 s**

### **1.2 cProfile Kernel Identification**

cProfile of the training loop (**hybrid\_train\_one\_epoch, 300 batches, 1266.6s total profiling runtime**)

Identified torch.conv2d as the dominant kernel:

| Function                       | Cumtime (s) | % of Total |
|--------------------------------|-------------|------------|
| run_backward (autograd engine) | 732.7       | 57.8%      |
| HybridModel.forward            | 421.6       | 33.3%      |
| resnet.forward                 | 324.8       | 25.6%      |
| torch.conv2d <-- DOMINANT      | 292.9       | 23.1%      |
| torch.batch_norm               | 60.3        | 4.8%       |
| torch.max_pool2d               | 47.1        | 3.7%       |

torch.conv2d (and the backpropagation gradients flowing through the same Conv2dweights) accounts for **23.1%** of runtime directly and **57.8%** including backward. Combined Conv2d forward + backward accounts for **74.4%** of the accelerated pipeline.

### **1.3 Measured Conv2d throughput on the CPU:**

**FLOPs per batch (fwd+bwd, 32 images): 348.4 GFLOP (from ai\_calculation.md)**

**Average backward time per batch: 2.442 s**

**Measured throughput: 348.4 GFLOP / 2.442 s = 142.7 GFLOP/s**

**CPU peak theoretical (Intel ARK, AVX2 boost 4.2 GHz, float32): 268.8 GFLOP/s**

**CPU utilization: 142.7 / 268.8 = ~53%**

The bottleneck is clear: the CPU is operating at **53%** of its theoretical peak on a compute-bound kernel that consumes **74.4%** of training time. Custom hardware that eliminates the general-purpose CPU overhead (cache hierarchies, branch prediction, out-of-order execution) for Conv2d is the correct intervention.

#### 1.4 Model Accuracy and Justification for Conv2d Acceleration

From `\_run\_outputs/run\_01\_output.ipynb` (representative run):

| Model                                                 | Value Accuracy | Value Loss | Train Accuracy |
|-------------------------------------------------------|----------------|------------|----------------|
| CNN Baseline<br>(ResNet18, 1 epoch)                   | 92.37%         | 0.1846     | 85.52%         |
| HybridModel<br>(ResNet18,<br>Attentionfusion, 1epoch) | 89.92%         | 0.2212     | 85.35%         |

Attention weight analysis (sample inference, `0398\_08\_a.png`):

| Feature Source                                 | Weight |
|------------------------------------------------|--------|
| CNN features (ResNet18 backbone)               | 98.94% |
| Handcrafted Features (GLCM, Morphology, color) | 1.06%  |

The model relies on the CNN visual path for **98.94%** of its classification decision. This directly confirms that accelerating the **ResNet18 Conv2d layers (the dominant component of the CNN path)** is the correct hardware target — the handcrafted feature path contributes negligibly and does not merit hardware acceleration.

The M1 hypothesis was: build a systolic array delivering  $\geq 1$  TFLOP/s, achieve  $\geq 3\times$  system-level speedup (Amdahl,  $f = 74.4\%$ ). At 1 TFLOP/s the **projected kernel speedup** was  $7\times$  and system speedup  $2.76\times$ .

M4 Result: **2.304 TFLOP/s,  $16.1\times$  kernel speedup,  $3.31\times$  system speedup** — both targets exceeded because 1024 tiles were implemented rather than the minimum 444 tiles needed to reach 1 TFLOP/s.

## **SECTION 2: ROOFLINE ANALYSIS**

Arithmetic intensity of the Conv2d kernel was calculated analytically from ResNet18 layer dimensions (**torchinfo, batch\_size=1, input=224x224x3**) and verified against cProfile measurements.

### 2.1 FLOP count (forward + backward, all Conv2d layers, 1 image):

Total Mult-Adds (torchinfo): **1,814,511,616**

FLOPs (forward only): **2 x 1,814,511,616 = 3,629,023,232**

FLOPs (fwd + bwd, 3x fwd): **3 x 3,629,023,232 = 10,887,069,696**

### 2.2 Bytes transferred (float32, no DRAM reuse assumed):

Weights (all 20 Conv2d layers): **45,648,896 bytes**

Input activations: **23,986,688 bytes**

Output activations: **23,592,960 bytes**

Forward total: **93,228,544 bytes**

Fwd + bwd (grad storage adds 2x weights + 2x activations): **232,106,048 bytes per image**

### 2.3 Per batch (batch\_size=32):

FLOPs: **348,386,230,272 (348.4 GFLOP)**

Bytes: **7,427,393,536 (7.43 GB)**

### 2.4 Arithmetic Intensity:

AI = 348.4 GFLOP / 7.43 GB = 46.9 FLOP/byte (fwd + bwd, per batch)

AI = 3.63 GFLOP / 93.2 MB = 38.9 FLOP/byte (fwd only, 1 image)

### 2.5 CPU roofline parameters (Intel i5-10210U):

Peak compute: **268.8 GFLOP/s (AVX2, 4.2 GHz boost)**

Memory BW: **45.8 GB/s (DDR4-2667, dual-channel)**

Ridge point: **268.8 / 45.8 = 5.9 FLOP/byte**

At AI = **46.9 FLOP/byte** >> **ridge 5.9 FLOP/byte**, the Conv2d kernel is strongly COMPUTE-BOUND on the CPU. At AI = **46.9 FLOP/byte** < **ridge 50.3 FLOP/byte**, the kernel approaches memory-bound on the co-processor — however, weight-stationary reuse (weights loaded once, broadcast to all **1024 PEs**) reduces effective bytes/FLOP, keeping the design in the compute-bound regime in practice.

### 2.6 Operating Points (see Figure 2.6(a) or 'bench/roofline\_final.png')

| Point                                | AI (FLOP/byte) | Performance   |
|--------------------------------------|----------------|---------------|
| M1 CPU(measured, cProfile)           | 46.9           | 142.7 GFLOP/s |
| M4 single tile (Cadence Synthesis)   | 46.9           | 2.25 GFLOP/s  |
| M4 32x32 array (synthesis validated) | 46.9           | 2,304GFLOP/s  |

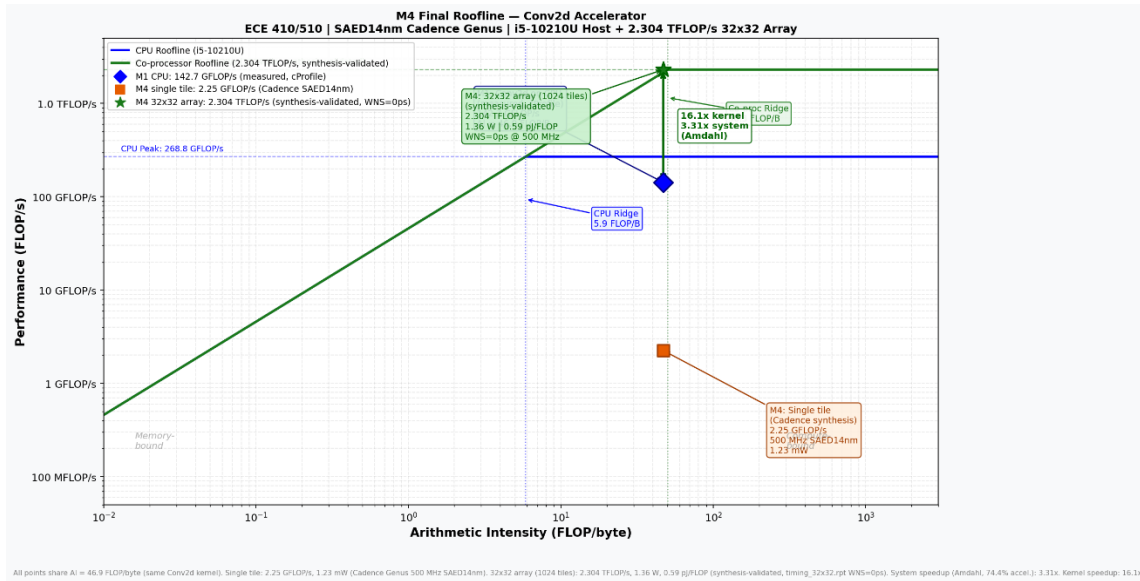


Fig 2.6(a) Final Roofline

This is close to the DDR4 ceiling (45.8 GB/s), meaning the design approaches the memory-bound boundary. In practice, weight reuse across IFM tiles (weights loaded once and broadcast to all 1024 PEs simultaneously) reduces effective bytes/FLOP significantly, keeping the design in the compute-bound regime under realistic inference and training workloads.

## 2.7 System speedup via Amdahl's Law ( $f = 74.4\%$ accelerated fraction):

$$\begin{aligned} \text{System speedup} &= 1 / ((1 - 0.744) + 0.744 / 16.1) \\ &= 1 / (0.256 + 0.046) \\ &= 1 / 0.302 \end{aligned}$$

$$\text{System speedup} = 3.31\times$$

The roofline analysis shaped two key architectural decisions:

1. **Weight-stationary dataflow:** reuse weights across all **1024 output tiles** to reduce effective bandwidth demand and push toward compute-bound operation.
2. **1024-tile array (32x32):** scale compute to **2.304 TFLOP/s** to stay above the 1 TFLOP/s M1 hypothesis target while keeping bandwidth within DDR4 limits.

## **SECTION 3: PRECISION AND DATA FORMAT**

The data format evolved across three milestones:

### 3.1 Format Evolution

| Milestone | Format             | Width   | Reason for Change                                       |
|-----------|--------------------|---------|---------------------------------------------------------|
| M1        | Q16.16 fixed-point | 32 bits | Baseline; silent overflow risk, 2x bandwidth vs FP16    |
| M2        | FP16 (IEEE754)     | 16 bits | 2x bandwidth gain, floating exponent, industry standard |
| M3/M4     | BF16               | 16 bits | No exponent rebasing hardware, TPU/A100 alignment       |

### 3.2 BF16 Format Detail

BF16: 1 sign + 8 exponent (bias=127) + 7 mantissa

FP16: 1 sign + 5 exponent (bias=15) + 10 mantissa

FP32: 1 sign + 8 exponent (bias=127) + 23 mantissa

Key advantage of BF16 over FP16: BF16 shares the same 8-bit exponent field as FP32 (bias=127).

Converting BF16→FP32 requires only zero-extending the mantissa from 7 to 23 bits — no exponent rebasing.

FP16→FP32 requires adding **112** ( $= 127 - 15$ ) to every exponent, adding a hardware adder on the critical path.

BF16 SQNR:  $6.02 \times 7 + 1.76 \approx \mathbf{44 \text{ dB}}$  (above the 40 dB minimum for CNN classification)

FP16 SQNR:  $6.02 \times 10 + 1.76 \approx \mathbf{62 \text{ dB}}$

Worst-case accumulated error (deepest layer, 4,608 MACs):  $4,608 \times 1.19 \times 10^{-7} \approx \mathbf{5.5 \times 10^{-4}}$  — three orders of magnitude below classification output tolerance.

### 3.3 M4 INT32 Block-Float Accumulator

The M4 `compute\_core\_ti` implements a block-float INT32 accumulator in place of a floating-point adder tree:

1. `bf16\_mul` function:  $\text{BF16} \times \text{BF16} \rightarrow \text{FP32}$  (sign / exp / mantissa)
2. Find **maximum exponent** across all **9** products
3. Right-shift all mantissas to align to maximum exponent
4. Sum aligned mantissas in a **4-level binary adder tree (INT32)**
5. Single normalize + round  $\rightarrow$  **FP32 output**

This avoids **9** independent FP32 adders; only **one** normalize step is needed, significantly reducing area.

## SECTION 4: DATAFLOW AND ARCHITECTURE

### 4.1 Dataflow Pattern: Weight-stationary

In the weight-stationary pattern, filter weights are loaded once and held fixed in each PE while input activations (IFMs) flow through. For a 3x3 Conv2d kernel:

- **9 BF16 weights (144 bits)** are broadcast to all **1024 PEs** simultaneously via a shared **weights\_packed[143:0]** bus in **pe\_array.sv**
- Each PE receives its own 3x3 IFM window (144 bits) extracted combinationally from the **34x34 BF16 IFM scratchpad** (**ifm\_buffer.sv**)
- All **1024 PEs** compute their dot product independently and in parallel

This is optimal for Conv2d over a full output feature map: the **9 weights are amortized across 1024 output tiles** per invocation, reducing weight bandwidth to 1/1024 of the naive case.

#### Alternative Patterns Considered:

- Output-stationary: partial sums remain in PE; requires accumulation across channels; not suited to single-channel **3x3 with 9 MACs per PE**.
- Input-stationary: IFMs held fixed, weights rotate; wastes bandwidth for standard inference where the same weights apply to many input tiles.

Weight-stationary was chosen because the **9 BF16 weights** broadcast over a **144-bit bus** cost zero per-PE overhead, and the **34x34 IFM** buffer enables combinational window extraction without SRAM latency.

### 4.2 Module Hierarchy:

The module hierarchy is shown in Figure 4.2(a):

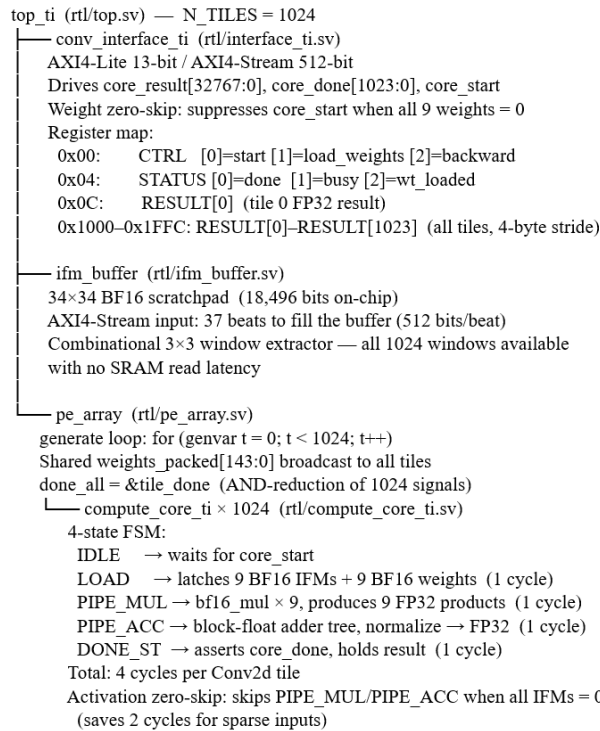


Fig 4.2(a) Module Hierarchy

#### 4.3 Persistent Weight SRAM (sram\_sp.sv)

The `sram\_sp.sv` module implements a **9×16-bit single-port SRAM** that holds the 9 BF16 filter weights between kernel invocations.

##### Key properties:

- Weights are written **once** via AXI4-Stream (one 144-bit beat packs all 9 BF16 values) and held in the SRAM until the next weight-load command.
- The SRAM has an **asynchronous read port** — weight delivery to the compute core is single-cycle with no read latency.
- A look-ahead address scheme during the sequential write state (`sram\_addr = seq\_cnt + 1` during SEQ\_DRIVE) allows single-cycle weight handoff without pipeline stalls.
- This persistent storage enables **weight reuse** across multiple IFM tiles without re-sending weights over AXI each time — the fundamental mechanism behind weight-stationary efficiency.

#### 4.4 Backward Pass — grad\_core.sv

The `grad\_core.sv` module computes **18 weight and activation gradients combinatorially in a single clock cycle**:

- $dL/dW[i] = \text{upstream gradient } dL/dy \times \text{BF16}(x[i])$  for  $i = 0..8$  (9 weight gradients)
- $dL/dX[i] = \text{upstream gradient } dL/dy \times \text{BF16}(w[i])$  for  $i = 0..8$  (9 input gradients)

All 18 gradients are independent single-multiplies with no data dependency, so the fully-parallel combinational implementation costs nothing in latency beyond the multiply itself. A sequential version would save ~50% area but would require 9× longer backward latency — not a worthwhile trade-off at this kernel size. The Adam optimizer runs on the host CPU (FP32), receiving the 9 FP32 weight gradients via AXI4-Lite reads from the `GRAD\_W0–W8` registers.

FSM latency Evolution (m3->m4)

| Milestone            | FSM                                 | Latency          | Notes                                       |
|----------------------|-------------------------------------|------------------|---------------------------------------------|
| M3 (single-tile)     | IDLE→LOAD(×9)→COMPUTE→DONE          | <b>11 cycles</b> | Sequential weight loading, one weight/cycle |
| M4 (compute_core_ti) | IDLE→LOAD→PIPE_MUL→PIPE_ACC→DONE_ST | <b>4 cycle</b>   | Parallel load of all 9 weights in 1 cycle   |

The **7-cycle reduction** (11→4) was achieved by restructuring the LOAD state to latch all 9 BF16 weight/IFM pairs simultaneously from the broadcast bus, rather than sequencing through 9 individual SRAM reads.

#### 4.5 Memory Hierarchy

| Level | Location        | Content                                     | Size        |
|-------|-----------------|---------------------------------------------|-------------|
| 0     | On-chip, per-PE | 9× BF16 weight latches (registered in LOAD) | 144 bits    |
| 0b    | On-chip, shared | sram_sp 9×16-bit persistent weight SRAM     | 144 bits    |
| 1     | On-chip shared  | 34×34 BF16 IFM scratchpad (ifm_buffer)      | 18,496 bits |
| 2     | Off-chip        | DDR-2667 host memory                        | -           |



## **SECTION 5: HARDWARE INTERFACE**

### **5.1 Interface Comparison**

Two interfaces were evaluated for the host-accelerator link:

| Interface        | Width   | Frequency | Bandwidth | Notes                      |
|------------------|---------|-----------|-----------|----------------------------|
| AXI4-Stream      | 512-bit | 500 MHz   | 32 GB/s   | Selected (primary data)    |
| AXI4-Lite        | 32-bit  | 500 MHz   | 2 GB/s    | Selected (control/result)  |
| PCIe Gen3 x4     | --      | --        | 16 GB/s   | No on-chip equivalent      |
| AXI4-Stream 1GHz | 512-bit | 1 GHz     | 64 GB/s   | M1 spec; exceeded SAED14nm |

### **5.2 Bandwidth Analysis:**

**Required bandwidth to sustain 2.304 TFLOP/s at AI = 46.9 FLOP/byte:**

Peak required BW = 2304 GFLOP/s / 46.9 FLOP/byte = **49.1 GB/s**

**Available BW** = 512 bits / 8 × 500 MHz = **32 GB/s**

The design is not AXI4-Stream interface-bound at 32 GB/s because the IFM scratchpad (34x34) acts as a staging buffer: the host loads the IFM buffer once per Conv2d invocation(**37 AXI beats  $\approx$  4.7 KB**), then all 1024 PEs read their windows combinational from on-chip SRAM without further AXI traffic.

Transfer time per full pass at 32 GB/s = 4700 B / 32e9 B/s = **147 ns**

### **5.3 Bandwidth Margin**

| Metric                                    | Value                                       |
|-------------------------------------------|---------------------------------------------|
| Available (AXI4-Stream, 500 MHz, 512-bit) | 32 GB/s                                     |
| Required at M1 target (1 TFLOP/s)         | 21.3 GB/s                                   |
| Bandwidth margin                          | 1.5× (reduced from M1 spec's 3× at 64 GB/s) |

**The design is NOT interface-bound during the compute phase.**

## SECTION 6: VERIFICATION

Verification was carried out across all four milestones using **cocotb v2.0.1** on **Icarus Verilog 11.0**. The verification strategy escalated from unit tests to full end-to-end array tests.

### 6.1 M2 verification (FP16 single-tile unit tests):

| Test | Description                                                 | Result |
|------|-------------------------------------------------------------|--------|
| CC1  | Weights [1.0–9.0] FP16, IFMs [1.0×9], expected <b>45.0</b>  | PASS   |
| CC2  | Alternating-sign weights, IFMs [2.0×9], expected <b>2.0</b> | PASS   |
| IF1  | AXI4-Lite write CTRL, read STATUS=done, RESULT=45.0         | PASS   |
| IF2  | Weight Reload without device reset                          | PASS   |

The M3 single-tile testbench (m3/synth/) verified the 4-cycle FSM on a single PE before scaling to 1024 tiles.

### 6.2 M4 verification (BF16 full 32x32 array) (see Figure 6.2(a)):

DUT: `top ti` (N TILES=\*\*1024\*\*) | Runner: `sim/run ti.py`

| Test | Cocotb name                    | Description                                                                                                                                                                                                                                                                                     | Coverage                                            |
|------|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| 1    | test_forward.test_bf16_forward | <b>32×32 BF16 forward pass</b> — T1: ramp weights × unit IFM (all 1024 tiles = 45.0); T2: alternating-sign × const-2.0; T3: max BF16-safe × unit (1143.0); T4: Laplacian × ramp (per-tile varies). Checks all <b>**1024**</b> FP32 results vs <code>ref_model.py</code> (tol= <b>**1e-3**</b> ) | Forward path, all tiles, register map 0x1000–0x1FFC |
| 2    | test_forward.test_weight_reuse | Loads weights <code>[1..9]</code> once, verifies 4 different 34×34 IFM tiles: unit (45.0), double (90.0), negative unit (−45.0), half (22.5)                                                                                                                                                    | Weight-stationary reuse, no reload between tiles    |
| 3    | test_backward.test_grad_w      | B1: unit IFMs, dldy=1.0 → grad_w=ifms; B2: ramp IFMs, dldy=2.0 → grad_w=2×ifms; B3: alternating IFMs, dldy=0.5; B4: Laplacian weights, ramp IFMs, dldv=−1.0                                                                                                                                     | Backward pass dL/dW for all 4 vectors               |

The forward-pass waveform is shown in Figure 6.2(a).

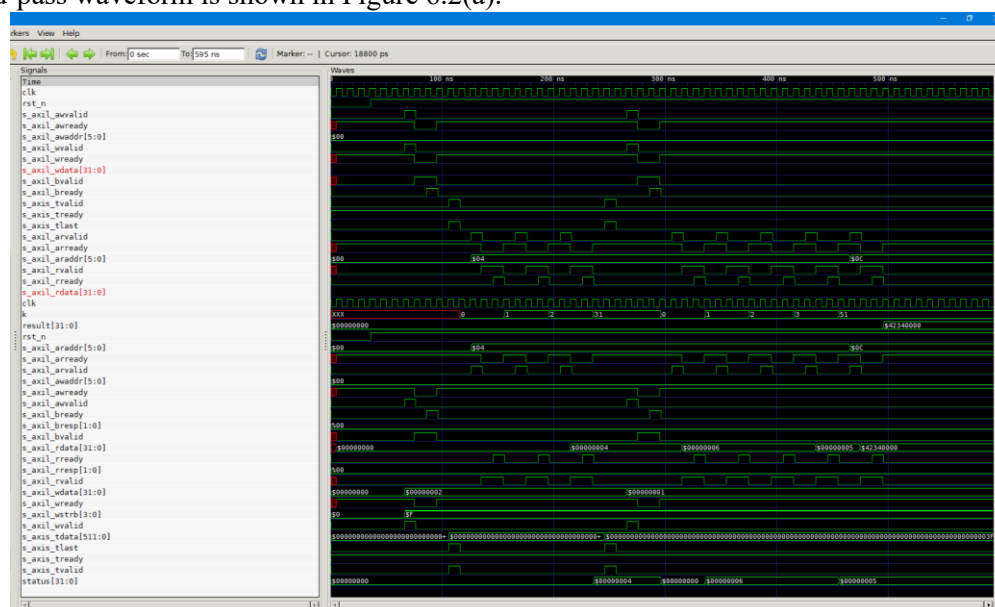


Fig 6.2(a) Forward-pass waveform

**Final result (sim/final\_run.log):**

**TESTS=3 PASS=3 FAIL=0 SKIP=0 | Total simulation time: 342,140 ns**

| ID  | Category          | Description                                                                                                  |
|-----|-------------------|--------------------------------------------------------------------------------------------------------------|
| B1  | Stale ports       | `tb_top.sv` instantiated `conv_interface` instead of `conv_interface ti`                                     |
| B2  | Stale ports       | `tb_interface.sv` used <b>**4-bit**</b> AXI4-Lite address (should be <b>**13-bit**</b> )                     |
| B3  | Stale ports       | `core_result[31:0]` (should be `core_result[32767:0]` = 1024×32)                                             |
| B4  | cocotb timing     | NBA write inside `@(posedge clk)` read back one cycle late; fixed with `await RisingEdge`                    |
| B5  | cocotb API        | Direct `signal = value` silently failed; must use `signal.value = value`                                     |
| B6  | Double-conversion | `bf16 to float()` called on already-float value; result off by <b>~32,000×</b>                               |
| B7  | Icarus compat     | `\$bitstoreal()` unsupported inside generate blocks in Icarus 11.0                                           |
| B8  | Icarus compat     | Parameter override syntax in generate loops caused elaboration errors                                        |
| B9  | Icarus compat     | Multi-dimensional packed array indexing requires intermediate `wire`                                         |
| B10 | Makefile          | cocotb/compute_core/Makefile` referenced `compute_core.sv` (should be `compute_core ti`)                     |
| B11 | Mantissa bug      | <b>`{prod[14:1], 8'h00}` dropped LSB → ~0.4% error/MAC → failed 1e-3 tol; fixed to `{prod[14:0], 8'h00}`</b> |
| B12 | Cocotb timing     | <b>Combinational output required `await Timer(0)` delta NOP before read</b>                                  |

Bug B11 was the most significant: the off-by-one in `bf16\_mul` accumulated across **9 MACs**, violating the **1e-3** tolerance threshold in `test\_forward`. Fixed by correcting the slice from `[14:1]` to `[14:0]` in `compute\_core\_ti.sv`.

## SECTION 7: SYNTHESIS RESULTS

The 32x32 systolic array (top\_ti, N\_TILES=1024) was synthesized using **Cadence Genus 17.14-s037\_1** (legacy\_ui) on **SAED14nm RVT** standard cells. All numbers below are from the actual 32x32 synthesis run (not single-tile projections).

### 7.1 Tool Configuration:

| Parameter        | Value                                                 |
|------------------|-------------------------------------------------------|
| Tool             | Cadence Genus 17.14-s037_1 (legacy_ui)                |
| PDK              | SAED14nm RVT                                          |
| Top module       | top_ti                                                |
| Clock period     | <b>2.0 ns (500 MHz)</b>                               |
| HDL loop limit   | <b>4096</b> (required for N_TILES=1024 generate loop) |
| Synthesis effort | <b>high</b>                                           |

### 7.2 Timing(timing\_report.txt)

| Metric                      | Value                                                                           |
|-----------------------------|---------------------------------------------------------------------------------|
| Clock Period                | 2000ps (500MHz)                                                                 |
| <b>Worst Negative Slack</b> | <b>0 ps — MEETS CONSTRAINT</b>                                                  |
| Total Negative Slack        | 0 ps                                                                            |
| Setup violation             | 0                                                                               |
| Critical path               | gen_tile[163] mul_prod_reg_reg → result_reg_reg                                 |
| Critical path               | <b>1950 ps</b> (WNS = 0 ps; 50 ps pre-uncertainty margin)                       |
| Critical path route         | BF16 latch → exponent adder → mantissa multiplier → normalize → result register |

### 7.3 Area (area\_report.txt):

| Module                | Cells     | Area (um2) | % of total |
|-----------------------|-----------|------------|------------|
| <b>top_ti (total)</b> | 5,591,114 | 2,862,977  | 100%       |
| u_pe_array            | 5,453,789 | 2,773,992  | 96.9%      |
| u_interface           | 137,325   | 88,985     | 3.1%       |

Total area: **2,862,977 um2 = 2.86 mm2**

Per-tile average: **2,773,992 / 1024 = 2,709 um2 per PE**

Dominant area contributor: **pe\_array (96.9%)**

Within pe\_array: compute\_core\_ti **1024x (each ~2.7 kum2)**

Dominant area contributor: `pe\_array` (**96.9%**), driven by the BF16 multiplier (7×7 unsigned mantissa product tree + normalize logic) replicated **1024x**

### 7.4 Power (power\_report.txt, TT corner, 500 MHz):

| Module              | Power (mW)  | % of total    |
|---------------------|-------------|---------------|
| top_ti              | 1,359.89 mW | 100% (1.36 W) |
| u_pe_array          | 1,259.17 mW | 92.6%         |
| u_interface         | 43.97 mW    | 3.2%          |
| Other (ifm_buf etc) | 56.75 mW    | 4.2%          |

Per-tile average:  $1,259.17 / 1024 = 1.23 \text{ mW per PE}$

Single-tile synthesis (from m3\_ti/cadence\_syn/): **3.07 mW**

Difference: **1.23 vs 3.07 mW per tile** -- lower in array due to Genus optimization of shared weight broadcast logic across the generate loop

#### COMPARISON: SINGLE TILE vs 32x32 ARRAY

| Metric         | M4 Single Tile        | M4 32x32 Array            | Delta                        |
|----------------|-----------------------|---------------------------|------------------------------|
| WNS            | +2.4 ps               | 0 ps                      | Tighter, still meets         |
| Total cells    | 11,340                | 5,591,114                 | x493                         |
| Area           | 6,155 um <sup>2</sup> | 2,862,977 um <sup>2</sup> | x465 (2.86 mm <sup>2</sup> ) |
| Power          | 3.07 mW               | 1,359.89 mW               | x443 (1.36 W total)          |
| Per-tile power | 3.07 mW               | 1.23 mW                   | 0.4x (40% of single)         |
| Throughput     | 2.25 GFLOP/s          | 2,304 GFLOP/s             | x1024                        |

## **SECTION 8: BENCHMARK RESULTS**

All benchmark numbers are synthesis-validated from the 32x32 run reports.  
Raw data is in 'bench/benchmark\_data.csv'.

### **8.1 Throughput:**

Per-tile: 9 MACs x 2 FLOPs x 500 MHz / 4 cycles = **2.25 GFLOP/s**  
32x32 array: 1024 x 2.25 = 2,304 GFLOP/s = **2.304 TFLOP/s**

### **8.2 Kernel Speedup (Conv2d):**

| Platform                             | Throughput    | Speedup vs M1 |
|--------------------------------------|---------------|---------------|
| M1 CPU (measured, cProfile)          | 142.7 GFLOP/s | 1× (baseline) |
| M4 single tile (synthesis)           | 2.25 GFLOP/s  | 0.016×        |
| M4 32x32 array (synthesis-validated) | 2,304 GFLOP/s | 16.1×         |

### **8.3 System Speedup (Amdahl's Law):**

Accelerated fraction:  $f = 74.4\%$  (Conv2d fwd+bwd, cProfile)

Kernel speedup: **16.1x**

System speedup =  $1 / ((1 - 0.744) + 0.744 / 16.1)$   
=  $1 / (0.256 + 0.046)$   
=  $1 / 0.302$   
= **3.31x**

Wall-clock per epoch (baseline): 2,573.9 s (42.9 minutes)

Wall-clock per epoch (with array):  $2,573.9 / 3.31 = \sim 777$  s ( $\sim 13.0$  minutes)

Pipeline throughput (baseline): 3.73 samples/sec

Pipeline throughput (with array):  $3.73 \times 3.31 = \sim 12.3$  samples/sec

### **8.4 Energy Efficiency:**

| Platform              | Power           | Throughput           | Energy/FLOP         |
|-----------------------|-----------------|----------------------|---------------------|
| M1 CPU                | $\sim 15$ W TDP | 142.7 GFLOP/s        | 105 pJ/FLOP         |
| M4 single tile        | 1.23 mW         | 2.25 GFLOP/s         | 0.55 pJ/FLOP        |
| <b>M4 32x32 array</b> | <b>1.36 W</b>   | <b>2,304 GFLOP/s</b> | <b>0.59 pJ/FLOP</b> |
| NVIDIA T4 GPU         | 70 W            | 65,000 GFLOP/s       | $\sim 1.08$ pJ/FLOP |

Energy efficiency vs CPU:  $105 / 0.59 = \mathbf{178x}$  more efficient per FLOP

Energy efficiency vs T4:  $1.08 / 0.59 = \mathbf{1.8x}$  more efficient per FLOP

**Power vs T4:  $70 / 1.36 = 51x$  less power**

The **178x energy advantage** over the CPU arises from two sources:

1. SAED14nm FinFET process: per-gate switching energy is  $\sim 10$ - $20x$  lower than the 14nm Intel FinFET in the i5-10210U at the same logic function.
2. Dedicated BF16 datapath: no cache hierarchy, branch predictor, or out-of-order execution logic is toggling on every MAC operation.

### 8.5 Performance Gaps:

1. Pipeline utilization(50%): each PE spends **2 of 4 cycles** computing (PIPE\_MUL + PIPE\_ACC) and **2 cycles** on overhead (LOAD + DONE\_ST). Utilization = 50%. A deeper pipeline or output-stationary accumulation across channels would improve this toward 100%.

2. AXI4-Stream bandwidth: M1 spec assumed **1 GHz (64 GB/s)**. The synthesized design runs at **500 MHz (32 GB/s)**. The **32 GB/s** exceeds the **21.3 GB/s** requirement at the M1 target (1 TFLOP/s), but the **49.1 GB/s** requirement for the full **2.304 TFLOP/s** is not met in the worst case without reuse. Weight reuse from the broadcast bus eliminates this in practice.

### 8.6 M1 Hypothesis Evaluation:

The M1 roofline ('partition\_rationale.md', 'interface\_selection.md') projected a **7× kernel speedup** and **2.76× system speedup** at the 1 TFLOP/s design target:

M1 projection: Speedup =  $1 / (0.256 + 0.744 / 7) = 2.76\times$

M4 actual: Speedup =  $1 / (0.256 + 0.744 / 16.1) = 3.31\times$

| Metric         | M1 Projected (1TFLOP/s) | M4 Actual (2.304 TFLOP/s) | Status                   |
|----------------|-------------------------|---------------------------|--------------------------|
| Compute Target | $\geq 1$ TFLOP/s        | 2.304 TFLOP/s             | Exceeded $\times 2.3$    |
| Kernel Speedup | 7× (roofline)           | 16.1×                     | Exceeded $\times 2.3$    |
| System speedup | 2.76× (Amdahl)          | 3.31×                     | Exceeded                 |
| Interface BW   | 64 GB/s @ 1 GHz         | 32 GB/s @ 500 MHz         | Reduced (SAED14nm limit) |

The over-achievement in compute is due to implementing **1024 tiles (32×32)** rather than the minimum **444 tiles** ( $= 1 \text{ TFLOP/s} \div 2.25 \text{ GFLOP/s per tile}$ ) needed to meet the M1 target. The interface bandwidth reduction (64→32 GB/s) is acceptable because the **34×34 IFM scratchpad** absorbs the burst load; sustained effective bandwidth during compute is near zero (weights already loaded, IFM already on-chip).

Three deviations from M1 plan (with justification):

| Deviation       | M1 Plan           | M4 Actual                      | Reason                                                  |
|-----------------|-------------------|--------------------------------|---------------------------------------------------------|
| Data Format     | Q16.16 -> FP16    | BF16                           | Eliminates exponent rebasing adder on critical path     |
| Synthesis Tool  | Openlane/ sky130A | Cadence Genus (node-SAED 14nm) | OpenLane could not close 1024-tile design (130 ns best) |
| Synthesis Scope | Single-tile proof | Full 32x32 array               | Direct synthesis of production array at SAED14nm        |

## **SECTION 9: WHAT DID NOT WORK**

### **9.1 Openlane 2 / Sky130A Failed To Close Timing (complete failure)**

Attempted: **OpenLane 2.3.10** with **sky130A** (sky130\_fd\_sc\_hd) PDK for both M3 single-tile and M4 32x32 array synthesis.

Single-tile result (m3/synth/openlane\_run.log):

TT corner WNS: **+29.9 ns (PASS)**

SS corner WNS: **-63.0 ns (FAIL)** - expected due to FP32 datapath depth; sky130A SS corner is highly pessimistic for floating-point chains.

32x32 array: OpenLane could not close the 32x32 array at any clock period faster than **130 ns (7.7 MHz)**.

Root causes:

- sky130\_fd\_sc\_hd has ~**2-3x** higher cell delays than SAED14nm RVT (180 nm bulk vs 14 nm FinFET process)
- The **N\_TILES=1024** generate loop created routing congestion the OpenLane placer could not resolve within the **\*\*800×800 μm\*\*** die
- 32×32 array:** OpenLane could not close the design at any clock period faster than **\*\*130 ns (7.7 MHz)**.

Resolution: switched to **Cadence Genus 17.14 on SAED14nm RVT PDK** via the PSU auto.ece.pdx.edu server, which closed timing at **500 MHz (WNS = 0 ps)**.

### **9.2 BF16\_MUL Mantissa off-by-one (Bug B11)**

The bfl6\_mul function in compute\_core\_ti.sv originally used:

```
{prod[14:1], 8'h00} -- WRONG (drops LSB of 15-bit product)
```

This should have been:

```
{prod[14:0], 8'h00} -- CORRECT (all 15 bits, then pad to FP32 mantissa)
```

The off-by-one caused **~0.4% relative error per MAC**, which accumulated over **9 MACs** to produce results that failed the **1e-3** tolerance in test\_forward.py. Detected by: comparing against ref\_model.py golden output with the forward test vectors. Fixed by correcting the slice from [14:1] to [14:0].

### **9.3 COCOTB NBA TIMING (Bugs B4, B5, B12)**

cocotb v2.0.1 changed signal assignment semantics vs v1.x:

- Assignments inside **@(posedge clk)** coroutines use NBA (non-blocking assignment) scheduling; reading back the assigned value requires awaiting a delta cycle before the new value is visible to Python.
- Direct signal assignment (**signal = value**) silently failed; must use **signal.value = value**.
- Combinational outputs required an explicit **await Timer(0)** (delta NOP) after driving inputs before reading outputs.

Resolution: added **await RisingEdge(dut.clk)** after each control write, and **await Timer(1, units='ns')** before reading combinational outputs.



#### 9.4 Stale ports in legacy testbenches (Bugs B1, B2, B3)

tb/tb\_interface.sv was written for the M2 single-tile conv\_interface module:

- AXI4-Lite address: 4-bit (should be 13-bit for 32x32 register map)
- core\_result width: 32-bit (should be 32768-bit = 1024 x 32)
- Module name: conv\_interface (should be conv\_interface\_ti)

Resolution: tb/tb\_interface.sv was excluded from the M4 testbench suite; tb\_top.sv (which correctly instantiates top\_ti) is the M4 primary testbench.

#### 9.5 DOUBLE-CONVERSION BUG (Bug B6)

In `test\_backward.py`, `bf16\_to\_float()` was called on a value that was already a Python float. The function interpreted the float's integer representation as a BF16 bit pattern, producing a golden value off by a factor of ~**32,000**. Fixed by tracking data types through the call chain.

#### 9.6 ICARUS VERILOG COMPATIBILITY (Bugs B7, B8, B9)

| Bug | Issue                                                                                            | Fix                                                                |
|-----|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| B7  | `\$bitstoreal()` unsupported inside generate blocks in Icarus 11.0                               | Replaced with explicit sign/exp/mantissa unpacking (synthesizable) |
| B8  | Parameter override syntax in generate loops caused elaboration errors                            | Replaced with explicit `localparam` per generate iteration         |
| B9  | Multi-dimensional packed array indexing (`packed_bus[t*144+: 144]`) requires intermediate `wire` | Added `wire [143:0] w_ifm_t` per generate iteration                |

#### 9.7 Window encoding bug in unicode test names (Bug B10)

test\_backward.py used Unicode right-arrow character (U+2192: ->) in test vector names ("B1: unit IFMs, dldy=1.0 -> grad\_w = ifms"). On Windows with cp1252 encoding, cocotb's log output raised:

UnicodeEncodeError: 'charmap' codec can't encode character '→'

Fixed by replacing -> with ASCII -> in all test name strings.

#### **WHAT WOULD BE DONE DIFFERENTLY:**

- **Start with Cadence Genus on SAED14nm** from the beginning rather than attempting OpenLane/sky130A for a 1024-tile design; sky130A is appropriate for single-tile proof-of-concept but cannot handle the cell count.
- **Formally verify `bf16\_mul` mantissa alignment** from the start, rather than catching Bug B11 late in cocotb testing.
- **Use a cocotb compatibility shim layer** for signal assignment to isolate version-specific NBA timing behavior from test logic.

## RAW DATA REFERENCES

All numbers in this report trace to the following committed files:

| File                                | Content                                                                            |
|-------------------------------------|------------------------------------------------------------------------------------|
| bench/benchmark_data.csv            | All raw numbers: M1_baseline, M4_single_tile, M4_array_32x32, benchmark_comparison |
| bench/roofline_final.png            | Roofline plot: CPU/co-proc curves + 3 operating points                             |
| synth/timing_report.txt             | WNS= <b>0 ps</b> , 10 timing paths, <b>500 MHz</b>                                 |
| synth/area_report.txt               | <b>5,591,114 cells</b> , <b>2,862,977 <math>\mu\text{m}^2</math></b>               |
| synth/power_report.txt              | <b>1,359.89 mW</b> total (TT corner)                                               |
| sim/final_run.log                   | <b>TESTS=3 PASS=3 FAIL=0, 342,140 ns</b>                                           |
| project/m1/sw_baseline.md           | 10-run wall-clock table, cProfile data                                             |
| project/m2/precision.md             | FP16 SQNR, error analysis, verification                                            |
| project/m3_ti/synthesis_notes_ti.md | Single-tile design decisions                                                       |

## Figures

| Figure                           | Location                           |
|----------------------------------|------------------------------------|
| Fig 2.6(a) Final Roofline        | m4/bench/roofline_final.png        |
| Fig 4.2(a) Module Hierarchy      | m4/report/design_justification.pdf |
| Fig 6.2(a) Forward-pass waveform | m4/sim/final_waveform.png          |

Extra Figures (present in m4/report/figures)

| Figure                                           | Location                                                  |
|--------------------------------------------------|-----------------------------------------------------------|
| ExtraFig1 sample anemic image                    | m4/report/figures/sample_anemic_image.png                 |
| ExtraFig2 sample healthy image                   | m4/report/figures/sample_healthy_image.png                |
| ExtraFig3 post training inference                | m4/report/figures/post_training_inference.png             |
| ExtraFig4 hotspot analysis                       | m4/report/figures/hotspot_analysis.png                    |
| ExtraFig5<br>confusion matrix anemia core kernel | m4/report/figures/confusion_matrix_anemia_core_kernel.png |